



طراحی سیستم‌های دیجیتال

بهار ۱۴۰۵

استاد: دکتر اجلالی

محسن صلاح - ۴۰۳۱۰۶۲۳۸

ضرب‌کننده‌ی اعداد مختلط

تمرین ۵ - سوال ۱

۱ هدف تمرین

در این تمرین، یک ضرب‌کننده‌ی اعداد مختلط پارامتری در Verilog طراحی می‌کنیم. این ماژول دو عدد مختلط A و B را دریافت می‌کند و بسته به سیگنال کنترل `conj`، یا حاصل ضرب معمولی $A \times B$ یا $A \times B^*$ را محاسبه می‌کند. تمام ورودی‌ها و خروجی‌ها به صورت signed در نمایش two's complement هستند.

۲ توضیح معماری

مدار یک pipeline دو مرحله‌ای synchronous است. در هر لبه‌ی بالارونده‌ی ساعت، ورودی‌های Ar ، Ai ، Br و Bi در رجیسترهای داخلی لچ می‌شوند. چهار combinatorial wire ($AiBr$ ، $ArBi$ ، $AiBi$ ، $ArBr$) حاصل ضرب‌های جزئی را از روی همین رجیسترها محاسبه می‌کنند، و در همان لبه‌ی ساعت Yi و Yr نیز آپدیت می‌شوند. از آنجایی که هر دو انتساب non-blocking هستند، محاسبه‌ی خروجی‌ها از مقادیر ذخیره شده‌ی چرخه‌ی قبل انجام می‌گیرد؛ بنابراین خروجی با تأخیر ۲ سیکل نسبت به ورودی ظاهر می‌شود.

۳ توضیح کد

۱.۳ ماژول `complex_multiplier (DSD_HW5_Q1.v)`

کد ماژول (بدون port declarations) به این صورت می‌باشد:

```
1 // registers for latching the inputs
2 reg signed [IN_WIDTH-1:0] Ar_reg, Ai_reg;
3 reg signed [IN_WIDTH-1:0] Br_reg, Bi_reg;
4 reg conj_reg;
5
6 // Intermediate products: 15-bit signed * 15-bit signed = 30-bit signed
7 wire signed [29:0] ArBr = Ar_reg * Br_reg;
8 wire signed [29:0] AiBi = Ai_reg * Bi_reg;
9 wire signed [29:0] ArBi = Ar_reg * Bi_reg;
10 wire signed [29:0] AiBr = Ai_reg * Br_reg;
11
12 always @(posedge clk) begin
13     // synchronous reset
14     if (rst) begin
15         Ar_reg <= {IN_WIDTH{1'b0}};
```

```

16     Ai_reg <= {IN_WIDTH{1'b0}};
17     Br_reg <= {IN_WIDTH{1'b0}};
18     Bi_reg <= {IN_WIDTH{1'b0}};
19     conj_reg <= 1'b0;
20     Yr <= {OUT_WIDTH{1'b0}};
21     Yi <= {OUT_WIDTH{1'b0}};
22 end else begin
23     // 1: latch inputs to the registers
24     Ar_reg <= Ar; Ai_reg <= Ai;
25     Br_reg <= Br; Bi_reg <= Bi;
26     conj_reg <= conj;
27
28     // 2: calculate outputs using values in the registers
29     if (!conj_reg) begin
30         Yr <= ArBr - AiBi;
31         Yi <= ArBi + AiBr;
32     end else begin
33         Yr <= ArBr + AiBi;
34         Yi <= AiBr - ArBi;
35     end
36 end
37 end

```

پارامترهای IN_WIDTH و OUT_WIDTH (با مقادیر پیش فرض ۱۵ و ۳۱) ابعاد ورودی و خروجی را تنظیم می‌کند. با فعال شدن rst، تمام registerها صفر می‌شوند. در غیر این صورت، ابتدا ورودی‌ها لچ می‌شوند و سپس، چون non-blocking assignment مقادیر register را صرفاً در انتهای سیکل اعمال می‌کند، خروجی‌ها از روی مقادیر سیکل قبل محاسبه می‌شوند. تعریف صریح ابعاد wireهای جزئی به ۳۰ بیت (دقیقاً کافی برای ضرب دو عدد ۱۵ بیتی signed) هم از ابهام در sign extension جلوگیری می‌کند و هم منطق combinatorial را از منطق sequential جدا می‌کند.

۲.۳ تست‌بنچ (DSD_HW5_Q1_tb.v)

task اصلی تست‌بنچ به این صورت است:

```

1 task run_test;
2     input signed [IN_WIDTH-1:0] in_Ar, in_Ai;
3     input signed [IN_WIDTH-1:0] in_Br, in_Bi;
4     input in_conj;
5     integer exp_Yr, exp_Yi;
6     begin
7         test_num = test_num + 1;
8         Ar = in_Ar; Ai = in_Ai; Br = in_Br; Bi = in_Bi; conj = in_conj;
9         @(posedge clk); @(posedge clk); #1;
10
11         if (in_conj == 1'b0) begin
12             exp_Yr = ($signed(in_Ar)*$signed(in_Br)) - ($signed(in_Ai)*$signed(
13                 in_Bi));
14             exp_Yi = ($signed(in_Ar)*$signed(in_Bi)) + ($signed(in_Ai)*$signed(
15                 in_Br));
16         end else begin
17             exp_Yr = ($signed(in_Ar)*$signed(in_Br)) + ($signed(in_Ai)*$signed(

```

```

        in_Bi));
16     exp_Yi = ($signed(in_Ai)*$signed(in_Br)) - ($signed(in_Ar)*$signed(
        in_Bi));
17     end
18
19     if (Yr == exp_Yr && Yi == exp_Yi) begin
20         passed = passed + 1;
21         $display("[PASS]_Vector_%0d:_...", test_num, ...);
22     end else begin
23         failed = failed + 1;
24         $display("[FAIL]_Vector_%0d:_...", test_num, ...);
25     end
26     end
27 endtask

```

این task ابتدا ورودی‌ها را set می‌کند، دو posedge صبر می‌کند (یک سیکل برای لچ شدن ورودی‌ها، یک سیکل برای آپدیت خروجی‌ها) و یک نانو ثانیه بعد از آخرین لبه نمونه برداری می‌کند تا از race condition جلوگیری شود. مقدار مورد انتظار داخل همان task با یک reference model محاسبه و با خروجی مقایسه می‌شود. قبل از شروع تست‌ها، یک چک مجزا برای حالت reset انجام می‌شود:

```

1  @(posedge clk);
2  rst = 0;
3  @(posedge clk); #1;
4  test_num = test_num + 1;
5  if (Yr === 0 && Yi === 0)
6      $display("[PASS]_Vector_%0d_(reset_check):_Yr=%0d,_Yi=%0d",
7              test_num, Yr, Yi);
8  else
9      $display("[FAIL]_Vector_%0d_(reset_check):_...", test_num);

```

بلافاصله بعد از آزاد شدن rst، بررسی می‌شود که Yr و Yi هر دو صفر هستند. تست‌های اصلی به ترتیب زیر هستند:

```

1 run_test(15'sd10, 15'sd5, 15'sd4, 15'sd3, 1'b0);
2 run_test(15'sd10, 15'sd5, 15'sd4, 15'sd3, 1'b1);

```

ضرب عادی و مزدوج برای اعداد با مؤلفه‌های مثبت؛ هر دو مُد conj با همان ورودی تست می‌شوند.

```

1 run_test(-15'sd8, -15'sd6, -15'sd3, 15'sd2, 1'b0);
2 run_test(-15'sd8, -15'sd6, -15'sd3, 15'sd2, 1'b1);

```

همین تست‌ها برای مؤلفه‌های منفی.

```

1 run_test(15'sd0, 15'sd0, 15'sd100, -15'sd50, 1'b0);

```

ورودی صفر برای A. خروجی در هر دو حالت conj باید صفر باشد.

```

1 run_test(15'sh3FFF, 15'sh3FFF, 15'sh3FFF, 15'sh3FFF, 1'b0);
2 run_test(15'sh4000, 15'sh4000, 15'sh3FFF, 15'sh3FFF, 1'b1);

```

مقادیر مرزی ۱۵ بیتی: 0x3FFF بزرگ‌ترین عدد مثبت (+۱۶۳۸۳) و 0x4000 کوچک‌ترین عدد منفی (−۱۶۳۸۴) است. این تست‌ها بزرگ‌ترین مقادیر خروجی ممکن را بررسی و ۳۱ بیت بودن خروجی را تأیید می‌کنند.

```

1 run_test(15'sh4000, 15'sh4000, 15'sh4000, 15'sh4000, 1'b0);
2 run_test(15'sh4000, 15'sh4000, 15'sh4000, 15'sh4000, 1'b1);

```

ضرب مرزی منفی در مرزی منفی — بزرگ‌ترین مقدار مطلق ممکن برای خروجی را ایجاد می‌کند ($Y_i =$ ضرب ۵۳۶۸۷۰۹۱۲ برای $conj=0$ و $Y_r = 536870912$ برای $conj=1$).

۴ اجرای کد

کد را compile و اجرا می‌کنیم. خروجی تست‌بنچ به این صورت می‌باشد:

```

mohsen@Quanta:/mnt/D/SUT/Term_4/DSD/HW/HW-5/DSD_HW5_Mohsen_Salah_403106238/Q1
/mnt/D/SUT/Term_4/DSD/HW/HW-5/DSD_HW5_Mohsen_Salah_403106238/Q1$ iverilog DSD_HW5_Q1.v DSD_HW5_Q1_tb.v
&& vvp a.out
VCD info: dumpfile DSD_HW5_Q1.vcd opened for output.

=== Complex Multiplier Testbench ===
[PASS] Vector 1 (reset check): Yr=0, Yi=0
-----
[PASS] Vector 2: A=(10, 5) B=(4, 3) conj=0 | Yr=25, Yi=50
-----
[PASS] Vector 3: A=(10, 5) B=(4, 3) conj=1 | Yr=55, Yi=-10
-----
[PASS] Vector 4: A=(-8, -6) B=(-3, 2) conj=0 | Yr=36, Yi=2
-----
[PASS] Vector 5: A=(-8, -6) B=(-3, 2) conj=1 | Yr=12, Yi=34
-----
[PASS] Vector 6: A=(0, 0) B=(100, -50) conj=0 | Yr=0, Yi=0
-----
[PASS] Vector 7: A=(16383, 16383) B=(16383, 16383) conj=0 | Yr=0, Yi=536805378
-----
[PASS] Vector 8: A=(-16384, -16384) B=(16383, 16383) conj=1 | Yr=-536838144, Yi=0
-----
[PASS] Vector 9: A=(-16384, -16384) B=(-16384, -16384) conj=0 | Yr=0, Yi=536870912
-----
[PASS] Vector 10: A=(-16384, -16384) B=(-16384, -16384) conj=1 | Yr=536870912, Yi=0
-----

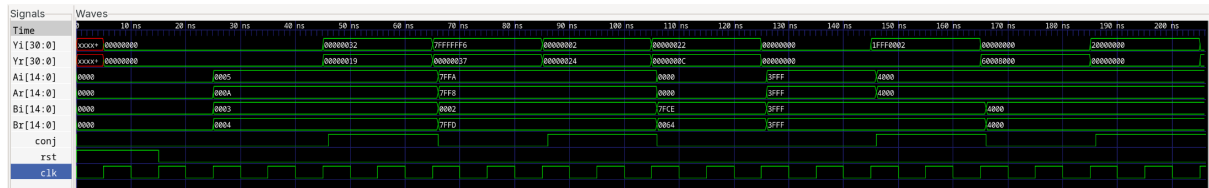
10 PASSED, 0 FAILED out of 10 tests.
DSD_HW5_Q1_tb.v:123: $finish called at 206000 (1ps)
mohsen@Quanta:/mnt/D/SUT/Term_4/DSD/HW/HW-5/DSD_HW5_Mohsen_Salah_403106238/Q1$

```

شکل ۱: خروجی تست‌بنچ

همان‌طور که مشاهده می‌شود، تمام ۱۰ تست با موفقیت پاس شده‌اند.

سیگنال‌ها را می‌توانیم در GTKWave نیز بررسی کنیم:



شکل ۲: شکل موج سیگنال‌ها در GTKWave

در موج‌نگار، تأخیر ۲ سیکل pipeline به وضوح قابل مشاهده است: بعد از اعمال ورودی‌ها، خروجی دقیقاً دو لبه‌ی ساعت بعد آپدیت می‌شود.